ML710 Project

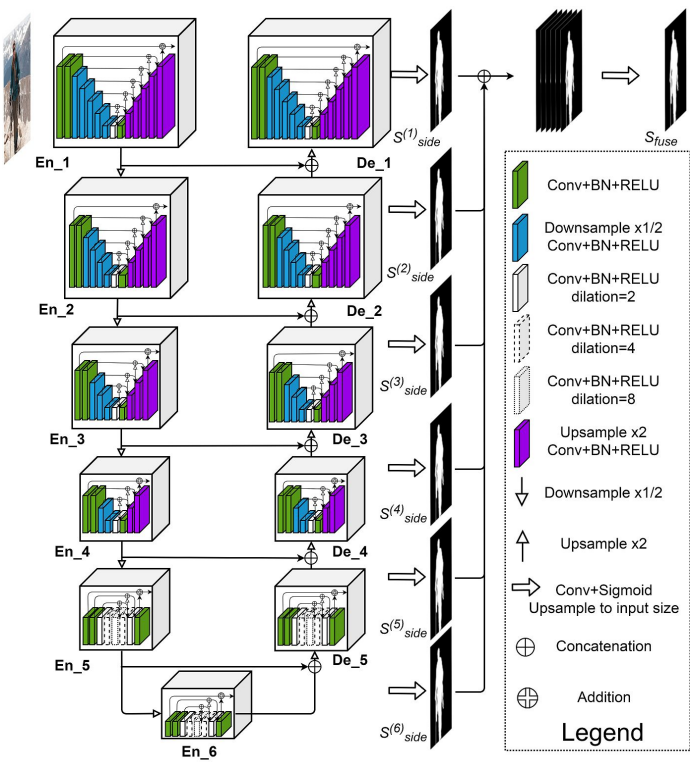
Maksym Bekuzarov
CV MSc, MBZUAI, 2022

Introduction

- Hardware: 1 node
  - GPU: NVIDIA A100-SXM4 (40GB) **x4**
  - CPU: AMD EPYC 7742, 64 physical cores, 256 virtual
  - RAM: 256 GB
- Task:
  - Train a Semantic Segmentation UNet-style CNN using Pipeline- and Data-parallelism

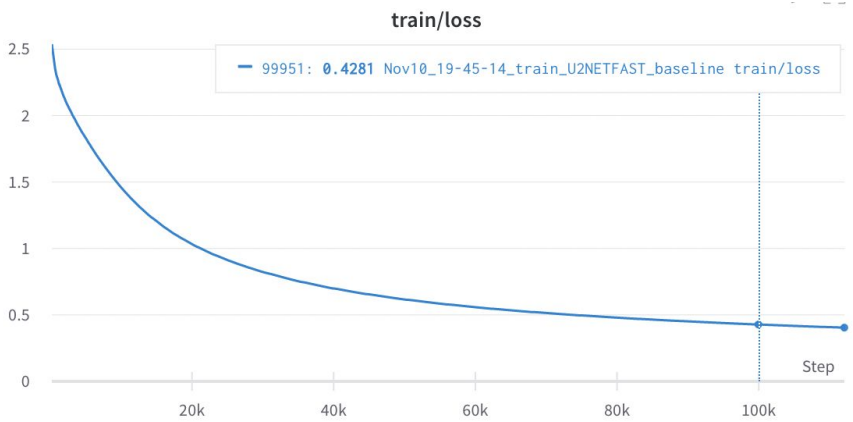
ML task

- Model: https://github.com/xuebinqin/U-2-Net
  - Basically UNet, but with nested U-style blocks and multiple side outputs (used for training with multi-scale loss)
  - **44M** parameters
- Dataset:
  - 3K images for training
  - 2470 images for testing (NOT USED IN THE PROJECT)
  - Sizes range from 720p to 4K+, so **high-resolution**



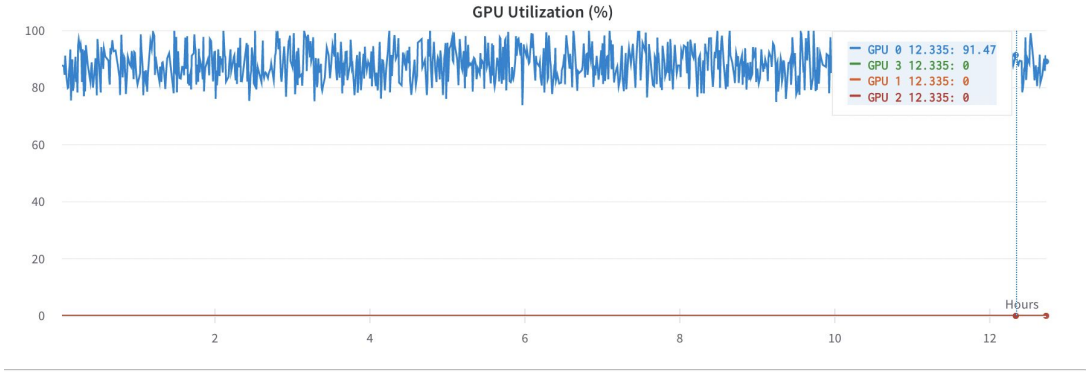
Baseline (1 GPU)

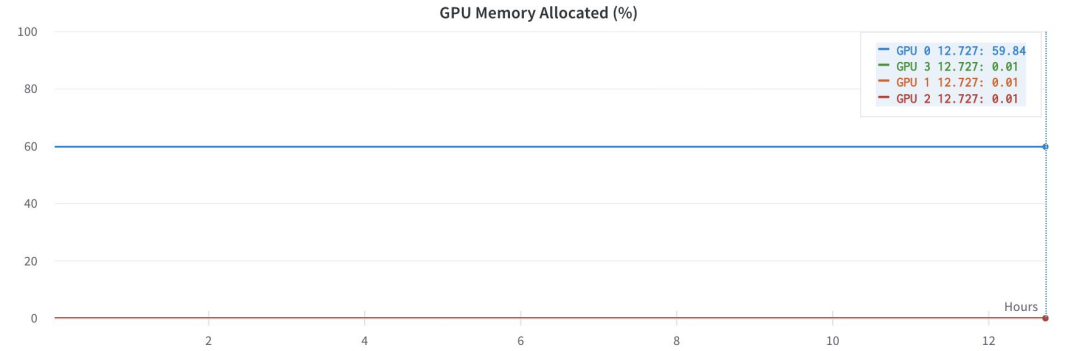
- Trained for 100K iterations
  - Image resolution: **1024**x**1024**
  - Batch size: **8** (default training config)
  - Wall time to 100K iterations: **11.4h**
  - Final loss: **0.428**



Baseline (1 GPU)

- Average GPU utilization: **90%**
- GPU memory allocated: **60%**





Pipeline Parallelism

Pipeline Parallelism

- Used gpipe: https://github.com/kakaobrain/torchgpipe
- Tried configurations for 2 and 4 GPUs
- Split the model down the middle for 2 GPUs, for 4 - split each part in 2.
- TODO:
  - Code to convert the model to Sequential
  - Portals (for skip connections)
  - Balancing attempts
  - Chose best, added DataParallel (just Torch DDP) on top
  - Practical considerations: just use DDP for all 4 GPUs
  - Findings? (maybe)

Pipeline Parallelism

- The way **gpipe** works is it requires us to have a “Sequential” model, and insert skip connections as special “layers” inside of it. These “layers” take the output of the previous layer and “stash” it (store it somewhere inside gpipe), and allow us to “pop” them in later parts of the model, wherever they are needed. This allows gpipe to move these tensors directly between “stash” and “pop” devices, skipping intermediate pipeline stages.

```
encoder_layers = []
for i, stage in enumerate([self.stage1, self.stage2, self.stage3, self.stage4, self.stage5]):
    ns = namespaces[i]
    encoder_layers.append(nn.Sequential(OrderedDict([
        ('encode', stage),
        ('skip_put', StashDecorated().isolate(ns)),
        ('down', nn.MaxPool2d(2, stride=2, ceil_mode=True))  # same as in the model
    ])))
```

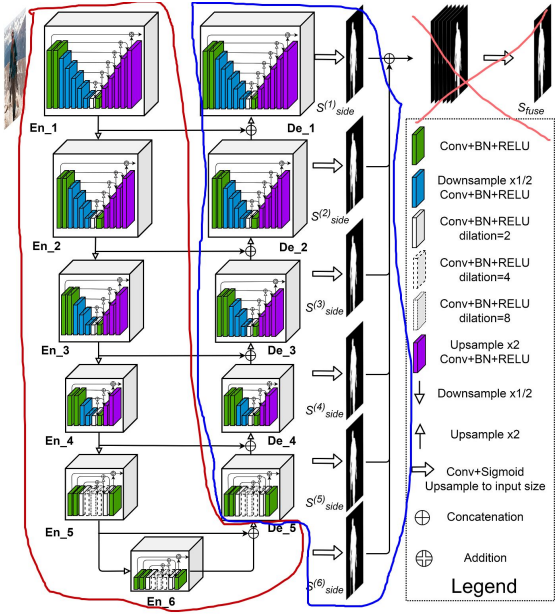
Encoder

```
decoder_layers = []
for i, staged in enumerate([self.stage5d, self.stage4d, self.stage3d, self.stage2d, self.stage1d]):
    ns = namespaces[depth - i - 2]
    stash_name = f'skip_side_{depth - i - 1}'
    decoder_layers.append(nn.Sequential(OrderedDict([
        ('up', nn.Upsample(scale_factor=2, mode='bilinear')),
        ('skip_get', PopCat().isolate(ns)),
        ('decode', staged),
        (f'skip_put_{depth - i - 1}', skippable(stash=[stash_name])(StashPure)(stash_name).isolate(side_skip_ns))
    ])))

last_decoder_layer = decoder_layers[-1]
del last_decoder_layer[len(last_decoder_layer) - 1]  # we actually don't need the last skip, as it === output
```

Decoder

Pipeline Parallelism: 2 GPUs equal split

- I had **38** layers, so 37 points where to cut the pipeline. I started with basic 19-19 equal split.
- The final side concatenation wasn't used in the code, so the output was just 6 side segmentation masks.
- All "side" convolutions were merged into 1 block (sequentially executed after last decoder stage)
- Microbatch size of **1** was used



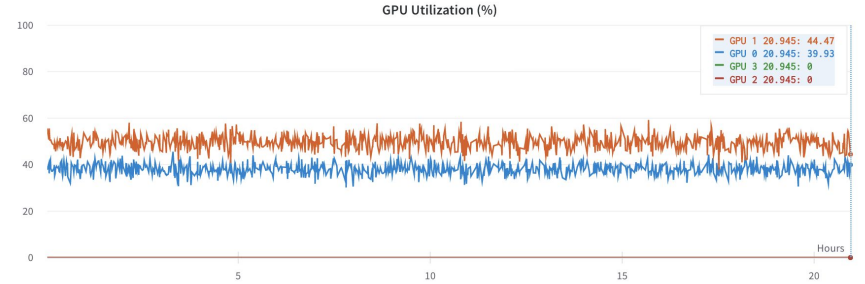

```python
@skippable(pop=[f'skip_side_{i}' for  i in range(2, 7)])
class SideOutput(nn.Module):
    def __init__(self, u2net_instance: U2NETFAST) -> None:
        super().__init__()
        self.sides = nn.ModuleList([u2net_instance.side1, u2net_instance.side2, u2net_instance.side3,
                                    u2net_instance.side4, u2net_instance.side5, u2net_instance.side6])

    def forward(self, staged_1_output):
        stages_output = [staged_1_output]
        for i in range(2, 7):
            s = yield pop(f'skip_side_{i}')
            stages_output.append(s)

        results = []
        for i, (side, skip_tensor) in enumerate(zip(self.sides, stages_output)):
            output = side(skip_tensor)
            output = F.upsample(output, size=(input_size, input_size), mode='bilinear')
            output = F.sigmoid(output)

            results.append(output)

        return tuple(results)  # from side1 to side6
```

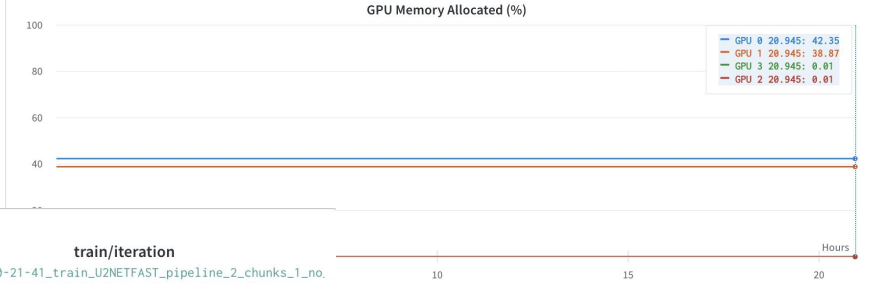
```
stem_0
encoder_0_encode
encoder_0_skip_put
encoder_0_down
encoder_1_encode
encoder_1_skip_put
encoder_1_down
encoder_2_encode
encoder_2_skip_put
encoder_2_down
encoder_3_encode
encoder_3_skip_put
encoder_3_down
encoder_4_encode
encoder_4_skip_put
encoder_4_down
bottleneck_0_encode
bottleneck_0_skip_put
decoder_0_up
decoder_0_skip_get
decoder_0_decode
decoder_0_skip_put_5
decoder_1_up
decoder_1_skip_get
decoder_1_decode
decoder_1_skip_put_4
decoder_2_up
decoder_2_skip_get
decoder_2_decode
decoder_2_skip_put_3
decoder_3_up
decoder_3_skip_get
decoder_3_decode
decoder_3_skip_put_2
decoder_4_0
decoder_4_1
decoder_4_2
side_0
```

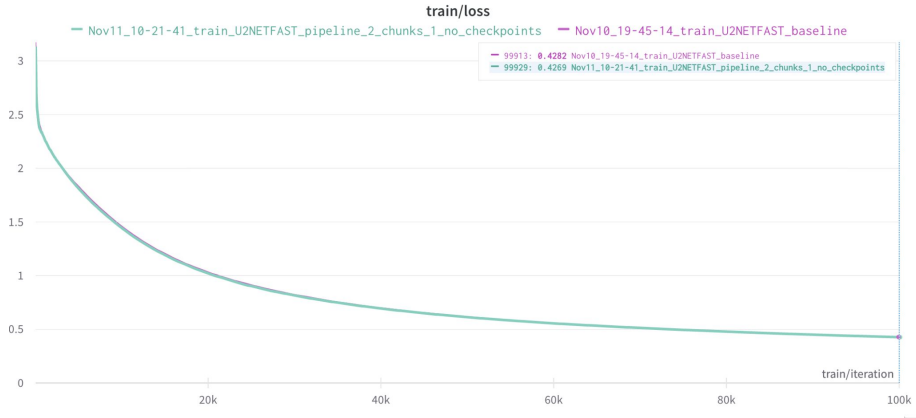
38 sequential layers

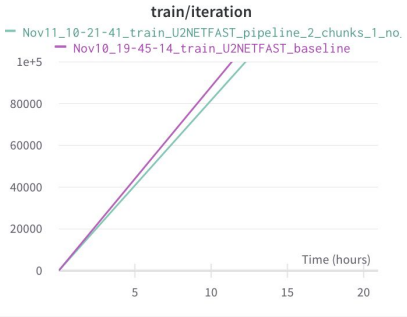
Pipeline Parallelism: 2 GPUs equal split

- Similar training wall time: **12.3h** (vs **11.3h** baseline, a +9% increase)
  - **As expected**, there is a communication overhead, but not that large
- Noticeable imbalance:
  - Average GPU utilization: **50% vs 40%**
  - GPU memory allocated: **42 vs 39%**
- No noticeable statistical efficiency drop: training loss exhibits the same behaviour
  - **As expected**, as by default Pipeline Parallelism does not affect statistical efficiency, except for when using BatchNormalization layers (in this case - negligible difference, **0.428** vs **0.427**, explainable by stochasticity)



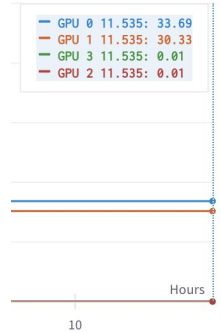


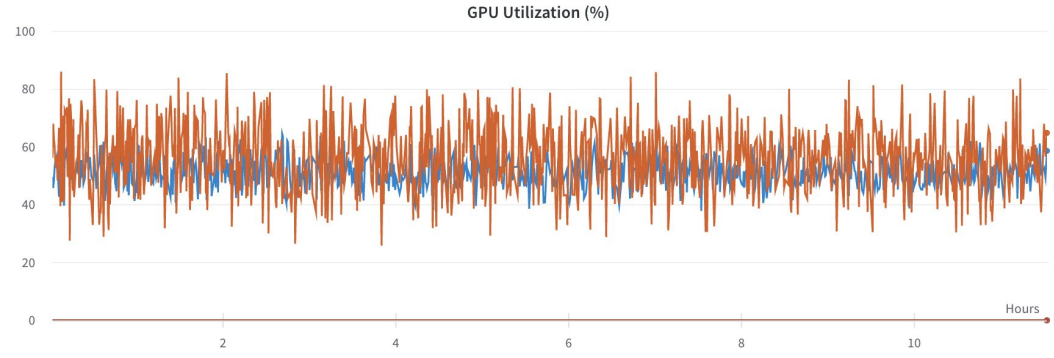


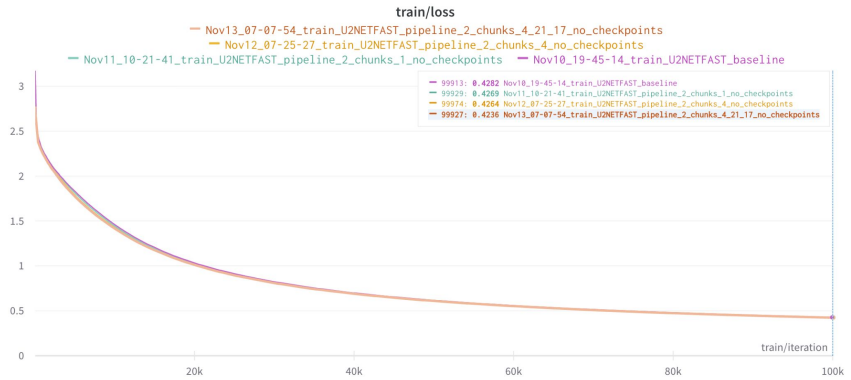


Pipeline Parallelism: 2 GPUs balanced split

- Using microbatch size of 4 (batch=8) reduced the wall time to **11.6h** (only 3% slower than baseline)
- Using a more balanced split (21-17) reduced the wall time to **11.3h** (same as baseline)
- Still no noticeable statistical efficiency loss, as expected
- Improved GPU usage:
  - Utilization: Both at **52%** on average (vs **50%** and **40%**), although more spikes
  - Memory: reduced to **34%** and **30%** (vs **40%** and **39%**)

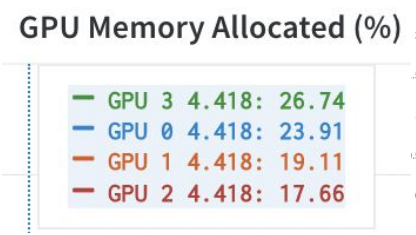




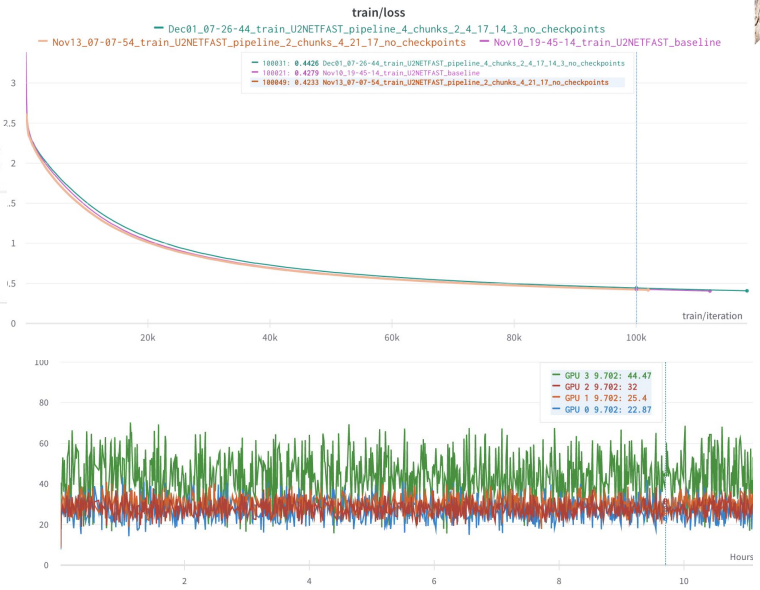


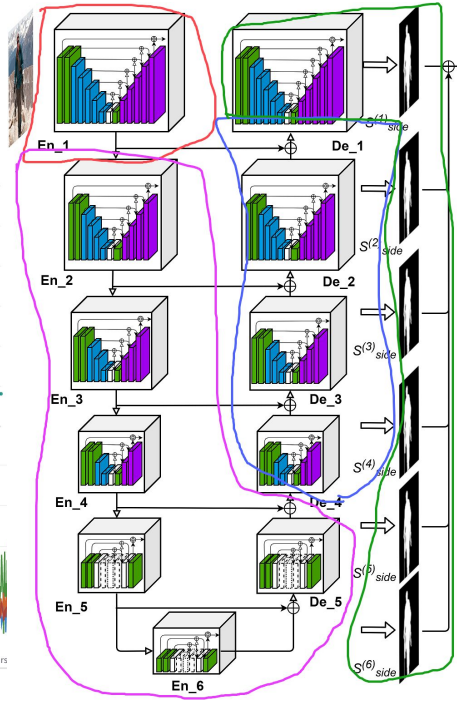
Pipeline Parallelism: 4 GPUs balanced split

- Took **6 attempts** to reasonably balance
- Average GPU utilization: 30%->30%->30%->50%
- Wall training time: **9.48h** (-16% vs baseline **11.3h**)
- A little bit higher loss (**0.443** vs **0.423** baseline), but no convergence problems (could be explained by randomness factor)



Even lower memory consumption, but start hitting diminishing returns





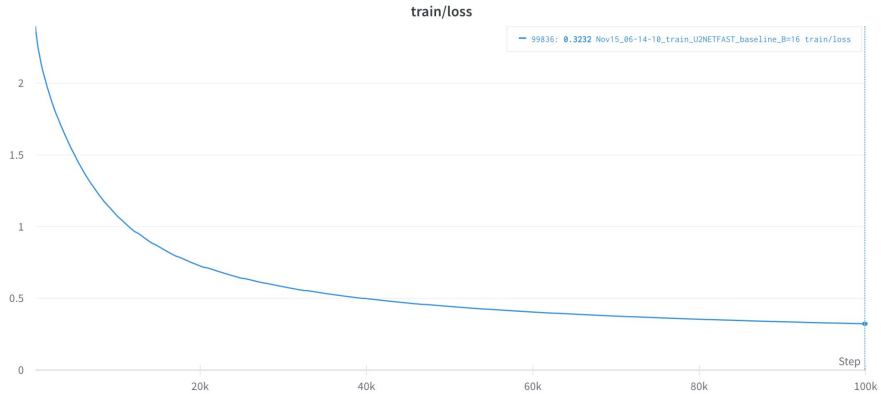

```
stem_0
encoder_0_encode
encoder_0_skip_put
encoder_0_down
encoder_1_encode
encoder_1_skip_put
encoder_1_down
encoder_2_encode
encoder_2_skip_put
encoder_2_down
encoder_3_encode
encoder_3_skip_put
encoder_3_down
encoder_4_encode
encoder_4_skip_put
encoder_4_down
bottleneck_0_encode
bottleneck_0_skip_put
decoder_0_up
decoder_0_skip_get
decoder_0_decode
decoder_0_skip_put_5
decoder_1_up
decoder_1_skip_get
decoder_1_decode
decoder_1_skip_put_4
decoder_2_up
decoder_2_skip_get
decoder_2_decode
decoder_2_skip_put_3
decoder_3_up
decoder_3_skip_get
decoder_3_decode
decoder_3_skip_put_2
decoder_4_0
decoder_4_1
decoder_4_2
side_0
```

Baseline 2 (maximum batch size) (1 GPU)

- Trained for 100K iterations
  - Image resolution: **1024**x**1024**
  - Batch size: **16** (soft cap, 24 and 32 crashed with OOM)

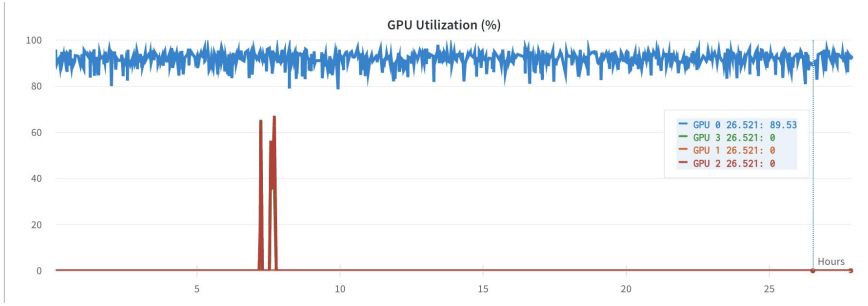
Nov15_06-12-56_train_U2NETFAST_baseline_B=24	failed	Add notes	max81(	2w ago	15s
Nov15_06-11-34_train_U2NETFAST_baseline_B=32	failed	Add notes	max81(	2w ago	15s

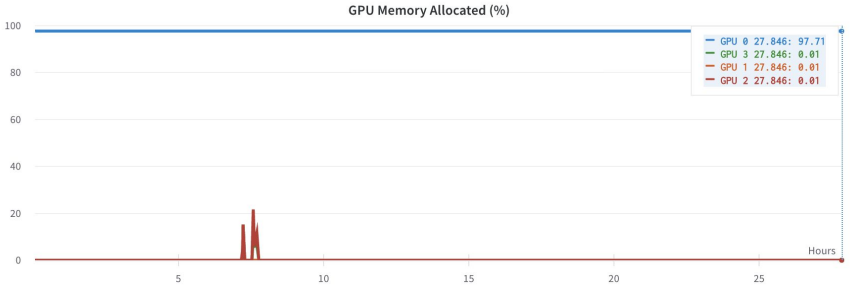
  - Wall time to **500K** processed samples:
    - **8.33h**
  - Loss @ 500K processed samples:
    - **0.5671**



Baseline 2 (maximum batch size) (1 GPU)

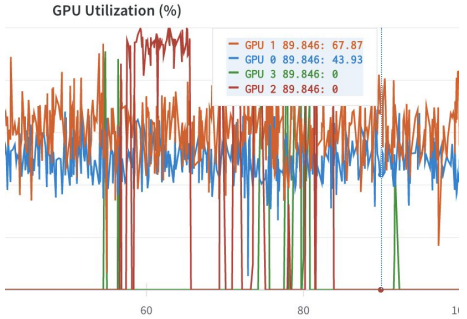
- Average GPU utilization: **90%**
- GPU memory allocated: **97%**

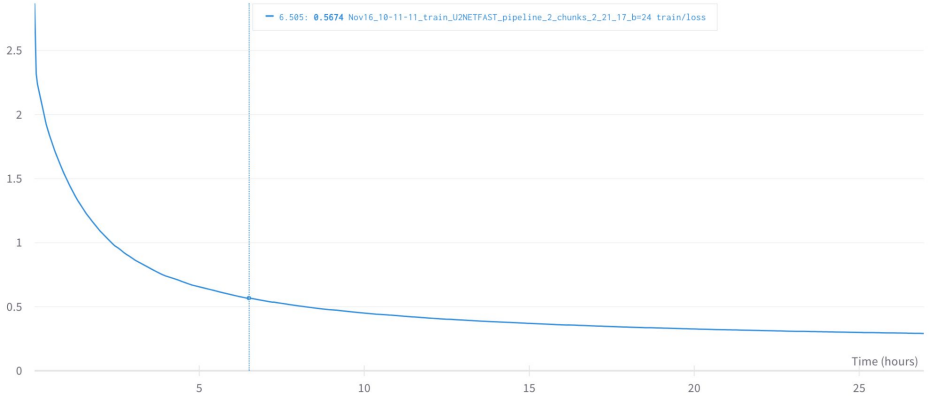


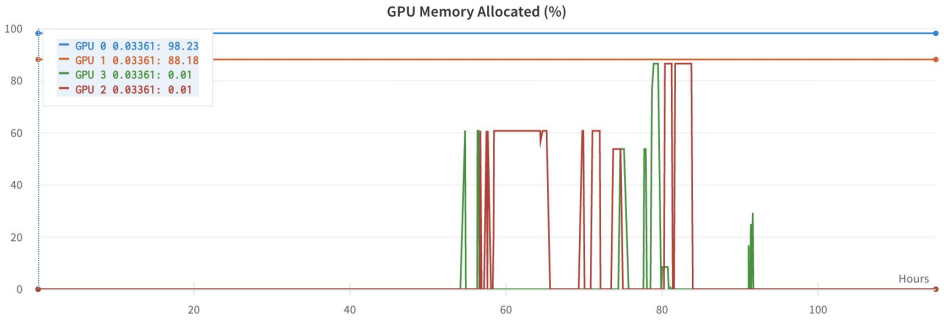


Pipeline (maximum batch size) (2 GPUs)

- Batch-size: 24 (50% more than 1 GPU)
- Average GPU utilization: **62%>50%**
- GPU memory allocated: **98%->88%**
- Wall time to **500K**: **5.72h** (-31.3% vs baseline **8.33h**)
- Loss @ 500K: **0.6087** (+0.042 vs baseline **0.5671**)
- Wall time to baseline **0.5671** loss: **6.5h** (-22% vs baseline **8.33h**)

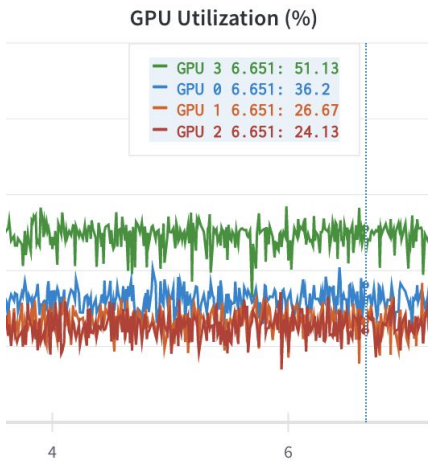


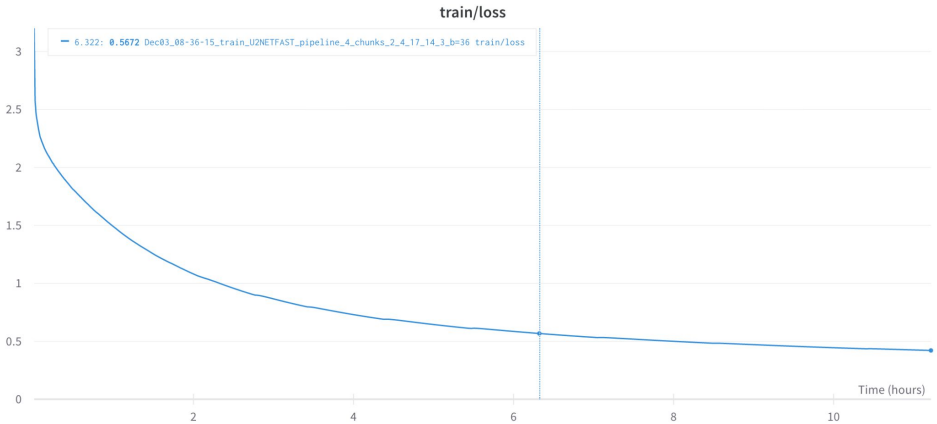


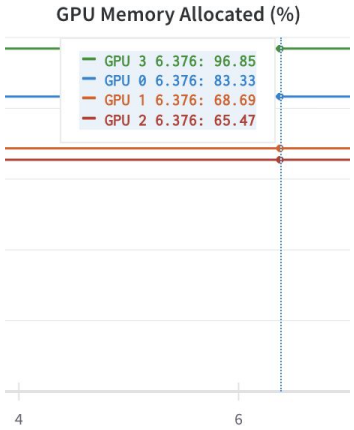


Pipeline (maximum batch size) (4 GPUs)

- Batch-size: 36 (125% more than 1 GPU)
- Average GPU utilization: **35%->25%->25%->50%**
- GPU memory allocated: **83%->69->65%->97%**
- Wall time to **500K**: **4.98h** (-40.2% vs baseline **8.33h**)
- Loss @ 500K: **0.6464** (+0.0793 vs baseline **0.5671**)
- Wall time to baseline **0.5671** loss: **6.32h** (-24% vs baseline **8.33h**)

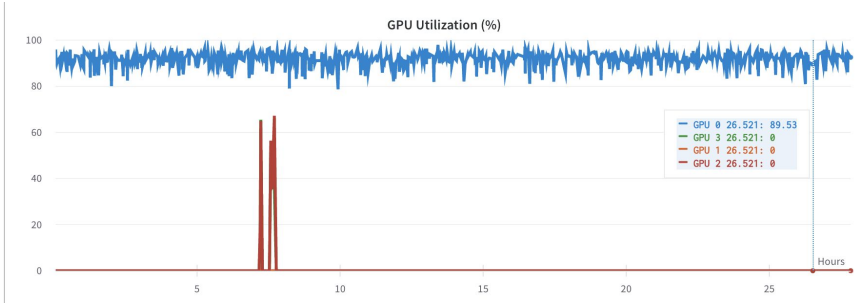


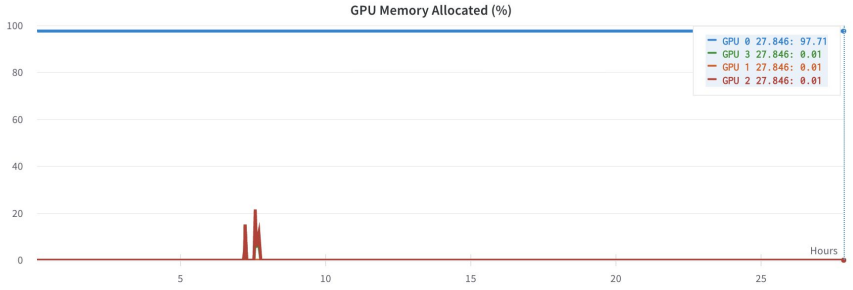




Pipeline (maximum batch size) (4 GPUs)

- Average GPU utilization: **90%**
- GPU memory allocated: **97%**



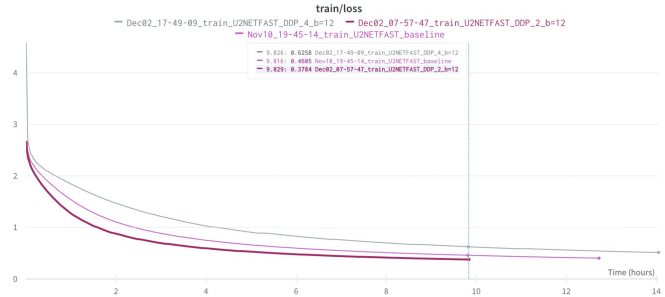


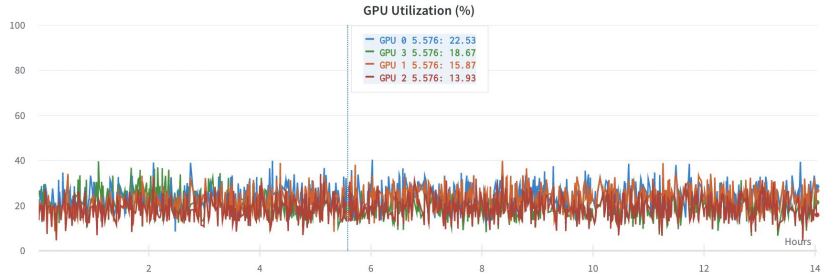
DDP (maximum batch size) (2 GPUs)

- Used PyTorch Distributed Data Parallel
- Batch-size (per GPU): **12**, not 16 (had OOM with 16 likely due to computation overhead)
  - So effective ML-step batch size=24
- Average GPU utilization: **78%>77%**
  - **Synchronization cost is noticeable, but not too bad**
- GPU memory allocated: **92%->88%**
- Wall time to **500K**: **4.3h** (-48.2% vs baseline **8.33h**)
- Loss @ 500K: **0.5728** (+0.0057 vs baseline **0.5671**)
- Wall time to baseline **0.5671** loss: **4.36h** (-47.7% vs baseline **8.33h**)

DDP (maximum batch size) (4 GPUs)

- Batch-size (per GPU): **12**, not 16 (had OOM with 16 likely due to computation overhead)
  - So effective ML-step batch size=48
- Average GPU utilization: **~20%** for all 4 GPUs!
  - Likely the **stragglers** effect, **communication** is taking much more time than **computation**
- GPU memory allocated: **99%->88->88%->88%**
- Wall time to **500K**: **9.45h** (+13.4% vs baseline **8.33h**)
- Loss @ 500K: **0.6412** (+0.074 vs baseline **0.5671**)
- Wall time to baseline **0.5671** loss: **11.83h** (+42% vs baseline **8.33h**)
  - Increasing batch size is not alway a good idea
  - Increasing #GPUs leads to a large communication overhead





DDP + pipeline (maximum batch size) (2+2 GPUs)

- Batch-size (per GPU): **24**
  - Effective ML-step batch size: **48**
- Average GPU utilization: **~40%-60%** on all 4 GPUs
- GPU memory allocated: **92%->88%**
- Wall time to **500K**: **3.45h** (-58.6% vs baseline **8.33h**)
- Loss @ 500K: **0.6874** (+0.1203 vs baseline **0.5671**)
  - Same noticeable drop in statistical efficiency as in DDP (due to large batch size)
  - But WAY smaller communication overhead
- Wall time to baseline **0.5671** loss: **4.85h** (-41.8% vs baseline **8.33h**)
  - Actually takes MORE time to get to baseline loss than DDP 2

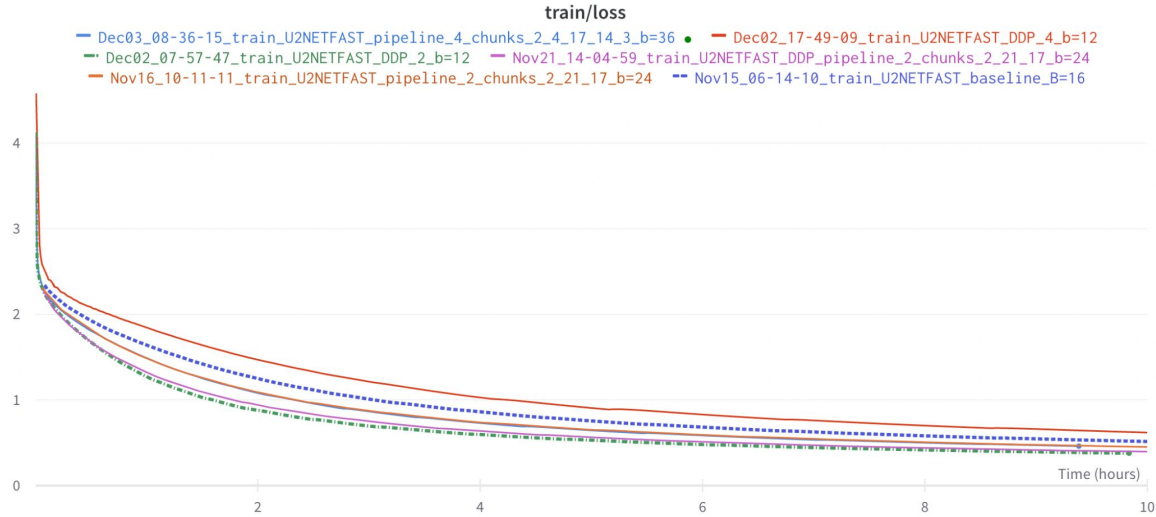
The best

Loss in 10h: DDP-2

Goodput: DDP Pipeline (2 + 2), 2nd best in Loss in 10h

WORST: DDP-4

Goodput might be a good metric, but calculating SE accurately is tricky



Detailed comparison

Experiment	Wall time to 100K iter, h	*Batch size	Loss @ 0	Loss @ 100K samples processed	Loss @ 500K samples processed	Throughput, item/s	*SE @ 0->100K samples	*SE @ 100K -> 500K samples	*Goodput	Loss @ 10h Wall
Baseline (b=8)	11.30	8	4.10	1.32	0.5461	19.66568338	0.02782	0.00192975	0.2925245821	0.456
Pipeline 2 GPUs (best)	11.3	8	4.10	1.28	0.5404	19.66568338	0.02818	0.001854	0.2953195674	0.4505
Pipeline 4 GPUs (best)	9.43	8	4.10	1.37	0.5673	23.56545305	0.02734	0.00199675	**0.3456669023**	**0.4045**
Baseline max (b=16)	26.6	16	4.10	1.36	0.5671	16.70843776	**0.02742**	0.00197725	0.245591061	0.5146
Pipeline 2 GPUs max (b=24)	27.5	24	4.10	1.41	0.6087	24.24242424	0.02689	0.00200575	0.3502515152	0.4511
Pipeline 4 GPUs max (b=36)	35.825	36	4.10	1.49	0.6464	27.91346825	0.02611	0.0021065	0.3938101884	0.4445
2 DDP	20.6	24	4.10	1.371	0.5728	32.36245955	0.02729	0.0019955	0.4738754045	**0.3783**
4 DDP	90.65	48	4.10	1.498	0.6412	14.70858614	0.02602	**0.002142**	0.2071116014	0.6194
Pipeline 2 GPUs + 2 DDP	33	48	4.10	1.51	0.6878	**40.4040404**	0.02591	0.002053	**0.5649090909**	0.3957
		*ML program effective batch size					* SE values multiplied by 10^3 for convenience			

[Detailed comparison table with SE, throughput and goodput metrics]

Comparison and findings

- Pipeline parallelism becomes harder to balance with more devices (2 attempts for 2 devices, 6 for 4).
  - It is **not obvious** how to split the layers between the devices at all! Especially on a **super-heterogeneous architecture** like UNet or U^2-net.
- For a UNet-style model took <12 hours to implement with gpipe, which is pretty quick.
- It significantly reduces memory usage (**60%** 1 GPU -> **~33%** 2 GPUs -> **~20%** 4 GPus)
- It allows for larger batch sizes (due to memory usage)
- It does not affect statistical efficiency* (kind of does, due to internal **Batch Normalization** 'magic' done by gpipe, but barely noticeably)
- Given a model that already fits in a single GPU, it is not very practical, as the speedup it provides is small (in our particular case).
  - Also, in such a settings it becomes hard to maintain high GPU utilization (**90%** -> **52%** -> **30%**), so the resources are wasted
  - If the main goal is to speed up the convergence of the ML program, and it already fits in a single GPU with reasonable batch size, using DDP is going to be both easier and way faster, but only for a **small number of GPUs** (in our case only 2)
  - Given these finding, pipeline parallelism will work great with a following setup: 1 node with **2-8 cheap GPUs** (is usually going to be way cheaper than 1 node with **1 expensive GPU**).
- DDP does not scale well with increased number of GPUs (due to stragglers effect taking place)
- It is important to start with simple techniques and later added more complex strategies ONLY if necessary (our 2+2 DDP pipeline is taking **more time to converge** than simple DDP 2)
- Pipeline parallelism seemed to scale better to more GPUs than DDP
- Maximizing the batch size is **not always a good idea** (statistical efficiency drops **surprisingly quickly** sometimes)
- **SE** is not always representative of true convergence behaviour (i.e. what will be the **final loss** after **N** wall hours?)

CODE

WandB experiments